

Octo Vault Internal Code Audit

Contents

1	Overview	2
2	Approach and Scope	2
3	Statistic Summary	3
4	Function and permissions	3
5	Findings	5
5.1	Summary	5
5.2	divide-before-multiply	5
5.3	missing-zero-check	6
5.4	calls-loop	6
5.5	costly-loop	7
5.6	redundant-statements	7
5.7	immutable-states	7

1 Overview

Octo Vault is a modular ERC4626-based vault system designed for secure, timelocked governance and composable yield strategies. Its architecture provides flexibility through modular components and plug-in execution hooks, allowing for controlled expansion and strategy composition.

The system consists of the following primary components:

- **BasedVault** — A base ERC4626 vault with optional execution hooks and timelocked governance. It manages parameters such as cap, timelock, and fee through defined roles (**admin**, **guardian**, **curator**). It can optionally act as a treasury when no hook is configured.
- **Market** — An ERC4626-compatible vault that delegates execution to whitelisted **Adapter** contracts. The **admin** manages adapter permissions, while **curators** perform supply, withdrawal, and harvesting operations.
- **Vault** — A meta-vault that allocates capital across approved Market vaults under a governance timelock. It supports market management actions such as add, veto, accept, or remove, and dynamically rebalances allocations through the **allocate()** function.
- **Hook** — An execution layer that extends vault functionality by integrating optional modules. It handles user operations including deposits, withdrawals, and transfers through standardized callbacks.
- **Modules** — Plug-in components that provide specialized operational logic:
 - **WithdrawModule**: Queues withdrawal requests by epoch and settles them pro-rata based on assets-per-share.
 - **ValuationModule**: Tracks overall portfolio value and accrues performance fees against a high-water mark.
 - **DepositModule (optional)**: Handles deposit queueing and claim processing when enabled.

Overall, Octo Vault’s architecture emphasizes security, modularity, and timelocked control, ensuring transparent governance and predictable contract behavior while maintaining composability for advanced yield strategies.

2 Approach and Scope

Auditors analyzed the smart contract’s functionality to identify potential vulnerabilities and assess its resistance to known attack vectors.

Slither Analysis

Using the Slither tool, auditors examined the code’s structure and patterns to detect possible weaknesses. The automated results were then manually reviewed to confirm findings and uncover additional issues.

Manual Review

A detailed line-by-line inspection was performed to evaluate the contract’s logic and security model. The audit also compared the implementation with the project’s documentation to ensure consistency with intended functionality.

Scope

The audit covered the core contracts that comprise the Octo Vault system. All reviewed source code was obtained from the official repository at commit hash 7850677c5cf00759e0902abd06c641390b2a820c at repository untangled-vault-v2. The analysis focused on security, correctness, and consistency with the project’s intended functionality.

The following components were included in scope:

- `contracts/Vault.sol`
- `contracts/BasedVault.sol`
- `contracts/Market.sol`
- `contracts/Hook.sol`
- `contracts/modules/WithdrawModule.sol`
- `contracts/modules/ValuationModule.sol`
- `contracts/modules/DepositModule.sol`

No external dependencies beyond OpenZeppelin libraries were included in this audit. Third-party adapters or integrations were not within the scope of this assessment.

3 Statistic Summary

Total number of contracts in source files: 7
Number of contracts in dependencies: 16
Source lines of code (SLOC) in source files: 793
Source lines of code (SLOC) in dependencies: 1242
Number of assembly lines: 0
Number of optimization issues: 1
Number of informational issues: 19
Number of low issues: 1

Use: OpenZeppelin-ERC20
ERCs: ERC165, ERC1363, ERC20, ERC4626

Name	# functions	ERCs	ERC20 info	Complex code	Features
Hook	18			No	
Market	102	ERC20, ERC4626	∞ Minting	No	Tokens interaction
Vault	100	ERC20, ERC4626	∞ Minting	No	Tokens interaction
DepositModule	9			No	
ValuationModule	15			No	
WithdrawModule	11			No	

Table 1: Summary of contracts and their key characteristics

4 Function and permissions

Function	Permission and condition
setModules	Vault's Admin
onDeposit	Vault
onWithdraw	Vault
settleWithdraw	Vault's Curator
claim	
updatePortfolioValue	Vault's Curator
setFeeRecipient	Vault's Admin
setFee	Vault
claimFee	Vault
onTransfer	Vault

Table 2: Function permission checks for Contract `Hook.sol`

Function	Permission and condition
setCap	Curator or Admin
vetoPendingCap	Guardian
acceptCap	
setTimelock	Curator or Admin
vetoPendingTimelock	Guardian
acceptTimelock	
setFee	Curator or Admin
vetoPendingFee	Guardian
acceptFee	
setHook	Admin
setCurator	Admin
setGuardian	Admin
transferAdmin	Admin
transfer	
transferFrom	

Table 3: Function permission checks for Contract `BasedVault.sol`

Function	Permission and condition
addAdapter	Admin
approve	Admin
removeAdapter	Admin
supply	Curator
withdraw	Curator
harvest	Curator
multicall	Curator

Table 4: Function permission checks for Contract `Market.sol` (inherits `BasedVault`)

Function	Permission and condition
totalAssets	
claimFee	
allocate	Curator
addMarket	Curator or Admin
vetoMarket	Guardian
acceptMarket	Curator or Admin
removeMarket	Admin

Table 5: Function permission checks for Contract `Vault.sol` (inherits `BasedVault`)

Function	Permission and condition
request	Hook
claim	Hook
settle	Hook
claimable	Hook

Table 6: Function permission checks for Contract `DepositModule.sol`

Function	Permission and condition
onDeposit	Hook
onWithdraw	Hook
updatePortfolioValue	Hook
claimFee	Hook
setFee	Hook

Table 7: Function permission checks for Contract `ValuationModule.sol`

Function	Permission and condition
request	Hook
claim	Hook require(orders[receiver].shares >= shares, Exceed requested shares)
settle	Hook require(shares <= totalWithdraw, Exceed total withdraw)
claimable	

Table 8: Function permission checks for Contract `WithdrawModule.sol`

5 Findings

5.1 Summary

- **divide-before-multiply** (1 result) – Low
- **missing-zero-check** (9 results) – Informational
- **calls-loop** (8 results) – Informational
- **costly-loop** (1 result) – Informational
- **redundant-statements** (1 result) – Informational
- **immutable-states** (1 result) – Optimization

5.2 divide-before-multiply

Impact: Low

- **ID-0:** `WithdrawModule.claimable(address)` performs a multiplication on the result of a division:
 - `settledShares = (remainShares * epochs[lastRequestId].fulfillment) / ONE_HUNDRED_PERCENT`
 - `settledAssets = (settledShares * epochs[lastRequestId].rate) / 10 ** shareDecimal`

File: `contracts/modules/WithdrawModule.sol#L153-L176`

Status: Acknowledged.

Recommendations The current implementation already utilizes basis point (bps) units, which helps mitigate most precision-related risks associated with divide-before-multiply operations. Nevertheless, developers should carefully review and verify all scaling factors, percentage constants, and decimal handling throughout the calculation process to ensure numerical accuracy and prevent rounding errors, especially in edge cases involving small values or large scaling differences.

Result: The team confirmed that bps-based arithmetic sufficiently minimizes the risk of significant precision loss. No functional issue was identified.

5.3 missing-zero-check

Impact: Informational

1. **ID-1:** `DepositModule.constructor(address)._hook`
lacks a zero-check on `hook = _hook`
File: `contracts/modules/DepositModule.sol#L13`
2. **ID-2:** `Hook.setModules(address,address,address)._valuationModule`
lacks a zero-check on `valuationModule = _valuationModule`
3. **ID-3:** `Hook.setModules(address,address,address)._depositModule`
lacks a zero-check on `depositModule = _depositModule`
4. **ID-4:** `Hook.setModules(address,address,address)._withdrawModule`
lacks a zero-check on `withdrawModule = _withdrawModule`
5. **ID-5:** `BasedVault.transferAdmin(address).newAdmin` lacks a zero-check on `admin = newAdmin`
6. **ID-6:** `ValuationModule.constructor(address)._hook`
lacks a zero-check on `hook = _hook`
7. **ID-7:** `WithdrawModule.constructor(uint256,address)._hook`
lacks a zero-check on `hook = _hook`
8. **ID-8:** `Hook.setFeeRecipient(address).newRecipient`
lacks a zero-check on `feeRecipient = newRecipient`
9. **ID-9:** `BasedVault.setGuardian(address)._guardian`
lacks a zero-check on `guardian = _guardian`

Status: Acknowledged.

Recommendations: Findings related to missing zero-checks occur primarily in initialization and admin-only functions. Because these functions are permission-restricted and cannot be invoked by untrusted users, omitting zero-address checks is acceptable as a deliberate gas-saving optimization. However, developers should ensure that all initialization parameters and administrative updates are validated during deployment or setup processes to prevent potential misconfiguration or deployment errors.

Result: Acknowledged. The team confirmed that these functions are restricted to trusted administrative contexts and that zero-address validation is handled operationally during deployment. No security risk was identified, and the decision to omit zero-checks remains acceptable under the current trust model. Future updates should continue to enforce strict deployment validation practices.

5.4 calls-loop

Impact: Informational

- **ID-10 to ID-17:** `Vault.allocate(Vault.Allocation[])` has external calls inside a loop at several points:
 - `BasedVault(allocation.market).approve(allocation.market,diff)`
 - `marketHook = address(BasedVault(allocation.market).hook())`
 - `BasedVault(allocation.market).withdraw(diff,address(this),address(this))`
 - `IERC20(asset()).approve(marketHook_scope_1,diff_scope_0)`

```

- IERC20(asset()).approve(allocation.market,diff_scope_0)
- BasedVault(allocation.market).deposit(diff_scope_0,address(this))
- BasedVault(allocation.market).approve(marketHook,diff)
- marketHook_scope_1 = address(BasedVault(allocation.market).hook())

```

File: contracts/Vault.sol#L90-L142

Status: Acknowledged.

Recommendations: External calls inside loops may pose reentrancy or gas consumption risks if the loop length or external behavior is not properly bounded. In this case, the loop iterates over a controlled list of allocation targets managed by trusted governance or internal logic, which limits exposure to such risks. Nevertheless, developers should:

- Ensure that the number of allocations is reasonably bounded to avoid potential gas exhaustion.
- Consider emitting events or adding safeguards to prevent excessive or unintended iterations.
- Review reentrancy protection mechanisms around external calls.

Result: The team confirmed that allocation loops operate over a fixed or governance-controlled set of markets, minimizing risk from unbounded iteration or external dependencies.

5.5 costly-loop

Impact: Informational

- **ID-18:** `Vault.removeMarket(address)` has costly operations inside a loop:

```

- markets.pop()

```

File: contracts/Vault.sol#L191-L202

Status: Acknowledged.

5.6 redundant-statements

Impact: Informational

- **ID-19:** Redundant expression `[shares]` in `Hook`

File: contracts/Hook.sol#L110

Status: Acknowledged.

5.7 immutable-states

Impact: Optimization

- **ID-20:** `Hook.vault` should be declared as immutable

File: contracts/Hook.sol#L21

Status: Acknowledged.